

Programmatic Access to the Nature's Palette Spectral Data Repository via a REST API and R Interface

Tridib Paul Turjo
Student ID: 202059523

Department of Computer Science
Memorial University of Newfoundland

Supervised by Dr. Adrian Fiech and Dr. Pierre-Paul Bitton

A dissertation submitted to the Department of Computer Science
in partial fulfillment of the requirements for the degree of
Bachelor of Science (Honours) in Computer Science

August 2026

Abstract

Nature's Palette is a web-based repository, developed at Memorial University of Newfoundland, for spectral data used in research on animal coloration. These datasets are costly to collect, access remains fragmented, and researchers commonly analyze spectral data in R. Nature's Palette previously supported only browser-based search and download, precluding a reproducible workflow.

This thesis addresses that gap with a REST API under `/api/v1`, built on the existing Node.js, Express, and MongoDB application, and an R package, `naturepaletteR`, that wraps the API for scripted use. The package allows researchers to search, retrieve, and download datasets directly from R, returning results as structured objects ready for analysis. The API reuses the repository's existing search, embargo, and download logic, keeping programmatic results consistent with the website. Automated testing, a parity comparison, and informal performance checks confirmed correct, consistent behaviour. The result is a repository supporting reproducible, script-based access, with authentication, documentation, and wider distribution identified as future work.

Acknowledgements

First and foremost, I would like to thank Memorial University of Newfoundland for giving me the opportunity to pursue my Computer Science Honours degree and to carry out this research. I am extremely grateful to my supervisors, Dr. Adrian Fiech, Associate Professor and Deputy Head of Graduate Studies in the Department of Computer Science, and Dr. Pierre-Paul Bitton, Associate Professor in the Department of Psychology, for their support and guidance throughout this Honours project. They generously gave their time, cooperation, and expertise, and they guided me through this thesis with great patience.

Finally, I would like to express my deep and sincere gratitude to my parents and my family for their continuous support and encouragement throughout my journey.

Table of Contents

Abstract	i
Acknowledgements	ii
Table of Contents	iii
List of Tables	v
List of Figures	vi
Chapter 1: Introduction	1
1.1 Background and Motivation	1
1.2 The Nature's Palette Repository	2
1.3 Problem Statement	4
1.4 Objectives	5
1.5 Scope and Limitations	6
1.6 Thesis Structure	7
Chapter 2: System Design and Development Approach	8
2.1 Design Principles	8
2.2 System Architecture and Request Flow	10
2.3 REST API Design Overview	12
2.4 R Client Design Overview	14
2.5 Development Approach	15
2.6 Deployment Environment	16
Chapter 3: Implementation	17
Part A: REST API	17
3.1 Core Endpoints	18
3.2 Catalogue Search	18
3.3 Submissions Search	21
3.4 Download	22
3.5 Dataset Metadata	24

3.6 Validation and Error Handling	24
Part B: R Client (naturepaletteR)	25
3.7 Package Structure and Dependencies	26
3.8 The np_search Object	26
3.9 Catalogue and Submission Search	28
3.10 Polygon Search and Client-Side Validation	29
3.11 Download, Metadata, and summary()	32
3.12 Distribution and Usage	33
Chapter 4: Testing, Evaluation, and Discussion	35
4.1 API Test Suite	35
4.2 R Package Test Suite	37
4.3 Comparison with the Web Interface	38
4.4 Performance Observations	42
4.5 Limitations	45
4.6 Future Work	48
Chapter 5: Conclusion	50
5.1 Summary of Work	50
5.2 Evaluation Outcomes	51
5.3 Contributions	51
5.4 Concluding Remarks	52
References	53

List of Tables

Table 3.8: Fields of the np_search object.....	27
Table 4.3: Summary of parity checks	41
Table 4.4: Response times (ms) for representative searches. Three runs per condition; median shown in the last column.....	44

List of Figures

Figure 1: High-level architecture of Nature's Palette with the additive REST API layer	10
Figure 2: Request flow for an API search or download.....	11
Figure 3: API folder structure	18
Figure 4: Example catalogue search response	21
Figure 5: naturepaletteR package folder structure	25
Figure 6: Terminal output showing a polygon search and a validation error message for a malformed polygon.....	31
Figure 7: Terminal output showing np_search_catalogue(), summary(s), np_search_submissions() and np_download().....	34
Figure 8: API test report showing pass/fail summary.....	36
Figure 9: testthat summary showing PASS/FAIL/SKIP counts	38
Figure 10: Catalogue search for genus Trogon, sex Male, specific epithet rufus, life stage Juvenile	39
Figure 11: Submission search by last name Sharier	40
Figure 12: Median response times for two search queries on the production server, by interface (API, web document request, R client). Derived from Table 4.4; indicative measurements, not a formal benchmark	45

Chapter 1: Introduction

1.1 Background and Motivation

Biological research increasingly relies on computational analysis of large datasets to investigate questions about evolution, behaviour, and ecology. In studies of animal coloration and visual communication, researchers collect spectral measurements that record how biological surfaces reflect light across different wavelengths. These measurements quantify colour properties objectively, which matters because different animal species perceive colour in fundamentally different ways depending on their eye structure and neural processing. Researchers apply this data to questions such as how vivid feather colours influence mate choice in birds, whether flower pigmentation matches the sensory capabilities of pollinators, or how prey animals achieve effective camouflage across varied habitats.

Collecting spectral data demands specialized equipment and substantial fieldwork or laboratory time, so each measurement session represents a significant investment of resources. Despite their scientific value, access to curated spectral datasets remains fragmented and inconsistent (Reichman et al., 2011). There is currently no standard practice for making such datasets broadly available once a study is published so that they can be discovered and reused (Wilkinson et al., 2016), and data are often shared informally rather than through organized repositories. Even where datasets are available,

they frequently require manual download through website interfaces that cannot be integrated into automated analysis pipelines. As a result, data management often falls back on ad hoc practices, which increase the likelihood of error and make it difficult to replicate or extend previous analyses (Powers & Hampton, 2019).

This problem is especially relevant for researchers who work in R, the statistical programming language most widely used for spectral data analysis in biology and ecology. Once spectral data are collected, researchers commonly process, analyze, and visualize them entirely within R, using purpose-built packages such as *pavo* (Maia et al., 2013). A repository that only supports manual, browser-based downloads breaks this workflow. A researcher must leave their analysis environment, search and download the data by hand, and then bring the files back into R before continuing their work. This is the same ad hoc step described above, and it grows more disruptive with each repeated, extended, or shared analysis. Programmatic access removes this step by letting researchers query and retrieve data directly from R, keeping data retrieval and analysis inside a single, reproducible workflow.

1.2 The Nature's Palette Repository

Nature's Palette is a web-based repository platform developed at Memorial University of Newfoundland to provide centralized storage and access for spectral data used in biological research. The system allows researchers to submit measurement files together

with descriptive metadata and provides other users with search tools to find and download relevant datasets. The goal is to let curated spectral data be discovered and reused, rather than recollected for each new study. The original repository was developed as graduate work at Memorial University (Akhter, 2020), establishing the curatorial pipeline for submitting spectral data, the metadata model used for discovery, and the core web application for search and download.

Subsequent honours and project work have extended the platform. Kamal (2020) developed Nature's Palette Validator, an offline cross-platform tool for checking metadata and raw spectral files before upload, which improves the submission side of the repository by helping researchers resolve validation issues locally, before their data ever reach the server. Khan (2023) added colour-space filtering to the web search interface, using dichromatic vision as a model case, extending how users can filter records in the browser. Together, these projects improve data ingestion, validation, and interactive search, but none of them provides scripted or programmatic access to search and download.

The repository follows a client–server architecture. It is hosted on Compute Canada's Arbutus cloud infrastructure, which runs the application server, database, and stored spectral data files. The web application itself is built with Node.js and Express on the

server side, with MongoDB used for data storage. Users interact with the system entirely through web pages rendered by the server.

The web interface supports several search and access features. Users can filter catalogue records by metadata fields, restrict results to a geographic region, and search by submission details such as researcher name or publication reference. Embargo rules govern when restricted data become publicly accessible, and matching results can be packaged into a downloadable archive. All these features, however, were available only through the browser, with no programmatic interface exposing them. The gap this creates, and how this thesis addresses it, is described next.

1.3 Problem Statement

The Nature's Palette web interface supports exploratory search and manual download well but does not provide programmatic access. A researcher who wants to repeat an analysis, update a dataset, or document how data were retrieved must re-enter the same search criteria through the website each time. That process cannot be saved as code, shared as a script, or run automatically as the repository grows.

In many scientific domains, programmatic access through an application programming interface, or API, is now standard practice (Norton, 2021), reflected in large public

biodiversity infrastructures such as GBIF (Telenius, 2011). An API lets researchers define queries in code, rerun analyses as datasets change, and share transparent retrieval steps alongside their publications. Without one, Nature's Palette remained hard to use inside scripted workflows, even though the underlying search and packaging logic already existed on the server.

This Honours project addresses that gap by providing a REST API under `/api/v1` and an R client package for programmatic search and download. The new interfaces reuse existing server-side behaviour so that programmatic results remain consistent with the website.

1.4 Objectives

The primary objectives of this Honours project are:

- To design and implement a language-agnostic REST API that exposes the existing search and download functionality of the Nature's Palette repository.
- To expose catalogue and submission search behaviour through the API so that, for equivalent query parameters, results and downloads remain consistent with the web interface.
- To develop an R-based interface that allows users to perform searches, retrieve structured results, and download datasets directly from R scripts.

- To improve the repeatability and automation of spectral data analyses by enabling script-based access to the repository.
- To evaluate the correctness, consistency, and indicative performance of the API and R interface in comparison with the existing web-based workflow.

1.5 Scope and Limitations

This project is limited to programmatic search and download for data already stored in Nature's Palette. It does not change the web submission or validation pipeline. It also does not add new biological search fields or new colour-space filters. The REST API and R package reuse the existing catalogue search, submission search, embargo rules, and download packaging behaviour.

Several boundaries follow from that scope. The API is additive. New routes are mounted under `/api/v1`, and legacy web routes are left unchanged. The API does not introduce new authentication or authorization for search and download. Public access follows the same rules already used by the website. Pagination is not implemented. Large catalogue queries return the full result set supported by the current server pipeline.

Detailed limitations of the delivered system and of the evaluation are discussed in Chapter 4.

1.6 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 presents the system design and development approach, including design principles, architecture, the REST API and R client design, and the development process used.
- Chapter 3 describes the implementation, covering the REST API (Part A) and the `naturepaletteR` package (Part B).
- Chapter 4 reports testing, evaluation, and discussion, including the API and R package test suites, comparison with the web interface, performance observations, limitations, and future work.
- Chapter 5 concludes the thesis, summarizing contributions and their significance for reproducible spectral data research.

Chapter 2: System Design and Development Approach

2.1 Design Principles

The design of this project was guided by four principles: mirror existing web behaviour in the API, add functionality without breaking the live website, keep new code modular and separate from legacy routes, and keep the R client a thin, dependable wrapper around the API.

First, the REST API was designed to mirror the functionality of the existing web interface rather than introduce new query semantics. No new biological fields or search logic were added. The API reuses existing server-side logic for metadata queries, geographic filtering, access control, and download packaging. This keeps programmatic results consistent with the website and reduces the risk of divergent behaviour.

Second, the API was added as a lightweight programmatic layer on the existing Node.js and Express application. It does not replace the web interface. New routes are mounted under `/api/v1`. Legacy route modules for paths such as `/search` and `/searchSubmission` were not repurposed. Selected existing controllers were extended to support JSON responses without removing their web handlers. The server endpoint was updated to

mount the new API routers. By adding new routers rather than rewriting existing route wiring, the risk of regressions in the web interface is reduced.

Third, new API code is organized modularly. New controllers are placed in `controllers/api/`, and new routes are placed in `routes/api/`, with an `api` prefix on filenames to distinguish them from legacy modules. This separation keeps the programmatic layer readable and maintainable and makes clear which parts of the codebase belong to the new API.

Fourth, the R package was designed to stay thin. It does not duplicate or reimplement server-side logic such as embargo filtering or query construction. Instead, it sends requests to the API and structures the response for use in R. Where the API's behaviour was incomplete for a scripted workflow, such as requiring a bounding box alongside a drawn polygon, validation and convenience logic were added on the R side rather than by changing the API's contract. This keeps the API as the single source of truth for search and download behaviour, while the R package remains easy to maintain and reason about.

Together, these principles shaped both the architecture described below and the implementation detailed in Chapter 3.

2.2 System Architecture and Request Flow

The system follows a client–server architecture, with the Nature's Palette repository hosted on Compute Canada's Arbutus cloud infrastructure. This project adds a REST API layer to that same server environment, so that clients such as the R package can call the repository programmatically alongside the existing web application.

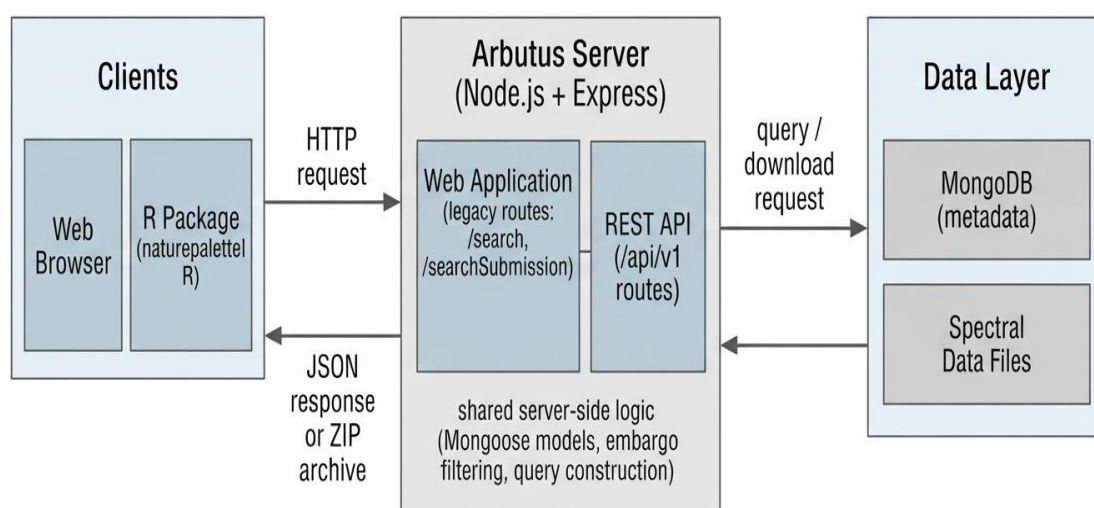


Figure 1: High-level architecture of Nature's Palette with the additive REST API layer

A typical API request flows through the system as follows:

1. A client sends an HTTP request to the server.
2. The request hits a new API route under /api/v1 and is passed to an API controller.
3. The controller calls query, grouping, or download helpers that mirror the web interface pipeline. These helpers use the same Mongoose models and shared utilities, including embargo filtering and query construction.

4. The server returns a JSON response for search and metadata requests, or a binary ZIP response for downloads.

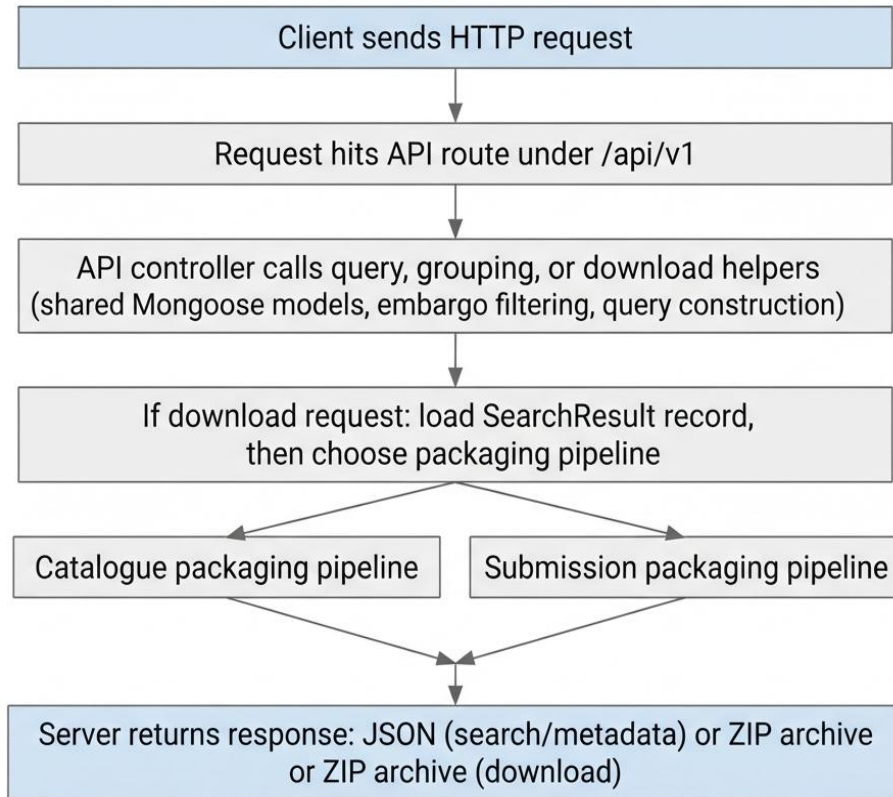


Figure 2: Request flow for an API search or download.

For downloads, the API controller loads the persisted search result and delegates to the appropriate catalogue or submission packaging pipeline, rather than assuming a single download path. Chapter 3 describes how this routing decision is made internally.

2.3 REST API Design Overview

The REST API exposes the repository's existing search and download behaviour through a small set of HTTP endpoints under `/api/v1`, following the REST architectural style (Fielding, 2000). Search and metadata requests return JSON. Download requests return a binary ZIP archive. The API does not define new biological fields or new query semantics; it provides a programmatic interface to behaviour that already exists on the server.

Four core endpoints were designed:

- GET `/api/v1/catalogue/search` — specimen-level catalogue search with filtering and grouping
- GET `/api/v1/submissions/search` — submission-level search
- POST `/api/v1/download` — download of a packaged archive by `SearchResultId`
- GET `/api/v1/datasets/{id}/metadata` — retrieval of structured metadata for a single dataset

Catalogue and submission search serve different workflows on the website, and the API preserves that distinction. Catalogue search supports the metadata and geographic filters available on the Search data page, drawing on the same style of structured biological metadata fields used by community biodiversity data standards such as Darwin Core (Wieczorek et al., 2012). Submission search supports the same submission filters used on

the Search by submission page. The supported fields for each endpoint are listed in Sections 3.2 and 3.3. On the API, catalogue parameters are sent as URL query strings, while the web form submits the same fields in an HTTP POST body.

Each successful search stores a SearchResult record. A client can run a search, read the returned searchResultId, and then request a download without repeating the original query. Download is exposed through a single endpoint for API clients, which routes internally to the appropriate catalogue or submission packaging pipeline; Chapter 3 describes the InputTerms mechanism used to make that decision.

Validation is applied where the API controllers check parameters directly, including minimum-length checks on submission search and structured error responses for missing identifiers. Catalogue geographic parameters follow the same parsing path as the web search, and invalid polygon geometry is not guaranteed to produce HTTP 400 in the current release; Chapter 3 describes this in more detail. Pagination through limit and offset was left as optional future work and is not implemented in the current catalogue search endpoint.

2.4 R Client Design Overview

The R client is provided by the package `naturepaletteR`, available at <https://github.com/tpturjo/naturepaletteR>. It is version 0.1.0 and is installed from GitHub rather than from the Comprehensive R Archive Network (CRAN), the main public repository for R packages. The package mirrors the website's search and download workflow, so that a researcher can run catalogue and submission searches, inspect the results, and download datasets from an R script instead of a browser. The package was kept deliberately small and thin, wrapping the API rather than reimplementing server-side logic. Chapter 3 describes its internal structure and dependencies in detail.

Both search functions return the same object type, a list with S3 class `np_search`. Using one object class for catalogue and submission search allowed `summary()` and `np_download()` to be written once and reused for both workflows. Chapter 3 describes the object's fields in full.

Five functions are exported for routine use, covering catalogue search, submission search, polygon formatting, dataset metadata lookup, and download, along with an S3 `summary()` method for readable inspection of search output. Their behaviour is described in Chapter 3.

As noted in 2.1, convenience logic was added on the R side where the API contract was incomplete for a scripted workflow, most notably for polygon search. There the client computes the required mapCoordinate bounding box automatically from a supplied polygon, so a user only needs to provide the polygon itself.

2.5 Development Approach

This project was developed iteratively, following an Iterative and Incremental Development (IID) approach built around weekly supervisor meetings. The system was built in stages: the REST API was designed and stabilized first, and the R client was developed afterward, on top of a working API.

Each stage followed the same cycle. A feature was implemented, tested manually with Postman (an HTTP API testing tool) or through exploratory checks, and then verified against the automated test suite once one existed. Work was reviewed at weekly meetings with the supervisor, and priorities or design decisions were adjusted based on that feedback before the next stage began.

Two examples illustrate this cycle in practice:

During integration testing of the download endpoint, a gap was found where POST `/api/v1/download` only invoked the catalogue download handler, so submission searches

did not produce a correct archive. This was corrected by adding routing logic to the API layer, without changing the legacy web controllers, and the automated download tests were then extended to cover both catalogue and submission cases.

The second example came from supervisor feedback after the R package's initial functionality was complete. The supervisor raised map polygon search as a feature that would add real value, since a polygon search run from R can be saved and rerun with identical results. After discussion, we agreed this was worth adding, and polygon search, along with coordinate validation to confirm a submitted polygon was well formed, was implemented in the R package as a result.

The core practices followed throughout were incremental delivery, one component working fully before the next began, continuous testing of each addition, and iteration driven by regular feedback rather than a fixed upfront plan.

2.6 Deployment Environment

The system was deployed on Compute Canada's Arbutus cloud infrastructure, consistent with the existing Nature's Palette repository. The REST API runs at <http://134.87.9.138/api/v1>, the single live deployment used by both the R package and by testing and development activity throughout the project.

Deploying to the server followed a consistent process throughout the project. Code was first validated locally using Postman, against a local development server, before being pushed to GitLab. Once deployed to Arbutus over SSH using Termius, the same requests were re-run with Postman against the live server, to confirm the deployed code behaved the same way it had locally. The automated test suite, described in Chapter 4, was then used as the ongoing method of verifying the live API as development continued.

The R package deploys differently, since it runs on a user's own machine rather than on a server. As described in 2.4, it is updated by reinstalling from GitHub, independent of the API server's release cycle.

Chapter 3: Implementation

Part A: REST API

This chapter describes how the REST API (Fielding, 2000) and R client were implemented, building on the design described in Chapter 2. Part A covers the API's endpoints and their internal behaviour. Part B covers the R client.

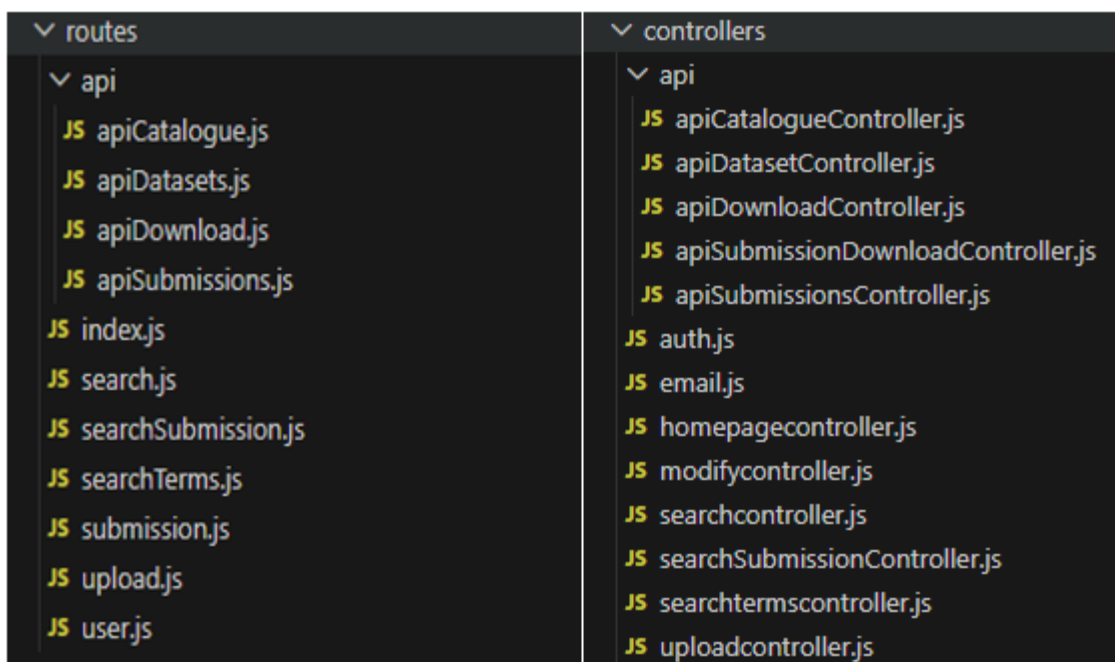


Figure 3: API folder structure

3.1 Core Endpoints

The four endpoints introduced in section 2.3 were each implemented by a dedicated API controller in `controllers/api/`, with routes defined under `routes/api/`. This section describes the internal behaviour of each endpoint. The implementation reuses the same Mongoose models, embargo rules, and download packaging logic used by the web interface. No new biological fields or search logic were introduced.

3.2 Catalogue Search

The catalogue search endpoint handles specimen-level searches. On the API, parameters are sent as URL query strings. The web form uses the same field names but submits them

in an HTTP POST body (Fielding et al., 2022). Internally, the endpoint uses a dedicated API helper that replicates the web catalogue pipeline. It follows the same query construction, embargo filtering, geographic filters, and result grouping logic. It shares the same Mongoose models.

The response is returned as JSON (Bray, 2017). It includes a search result ID, a totalMatchCount, and the grouped results. The totalMatchCount is the number of metadata rows returned by the database query. The results array is the grouped table shown to the client. When a polygon filter is used, points outside the polygon are removed from that results array after the query, so the number of displayed rows can be smaller than totalMatchCount. A download for the same SearchResultId is built from the metadata IDs stored for that query, so it follows the database result set rather than the polygon-filtered display list. This behaviour matches the existing web pipeline, but it is inconsistent: a user can see fewer rows than totalMatchCount suggests. Aligning totalMatchCount, the displayed results, and the download contents after polygon filtering is left as future work.

The supported biological filter fields include, among others, class, order, family, genus, specific epithet, infraspecific epithet, sex, lifestage, and patch, as enabled in the repository's search-term configuration, corresponding to terms defined by the Darwin Core biodiversity data standard (Wieczorek et al., 2012). Geographic filtering supports

both a bounding box and a polygon. A bounding box defines a rectangular geographic region using northeast and southwest latitude and longitude coordinates. It is applied directly at the database query level. A polygon defines an arbitrary multi-vertex geographic region. It allows irregular spatial filtering, but it requires post-query point-in-polygon validation in application memory.

Polygon search takes two parameters: `polyCoordinates`, describing the shape drawn by the user, and `mapCoordinate`, a bounding box surrounding that shape. Both are required for the grouping step to return results consistent with `totalMatchCount`. A client that supplies only `polyCoordinates` will receive a non-zero match count with an empty results array. The R client, described in Part B, computes `mapCoordinate` automatically so a user only needs to supply the polygon.

Pagination through `limit` and `offset` was left as optional future work and is not implemented in the current version of the catalogue search endpoint.

Embargo enforcement occurs at query time, consistent with the web interface behaviour. Only metadata linked to submissions where `embargo` is false, or where `embargo` is true and the release date is on or before the query day, is included in the search results.

```
134.87.9.138/api/v1/catalogue/search?genus=Trogon&sex=Male&specificepithet=rufus&lifestage=Juvenile
Pretty-print ☒

{
  "success": true,
  "searchResultId": "6a58a805805752cfdb9d01f1",
  "totalMatchCount": 2,
  "results": [
    {
      "SearchResultId": "6a58a805805752cfdb9d01f1",
      "TotalSearchResultCount": 2,
      "_id": "6a559bc9805752cfdb9d0087",
      "rawFileId": "6a559bc5805752cfdb9d0053",
      "genus": "Trogon",
      "specificepithet": "rufus",
      "infraspecificepithet": "rufus",
      "sex": "Male",
      "lifestage": "Juvenile",
      "patch": "Head",
      "latitudeList": [
        "E09~47.5615",
        "E10~44.6488",
        "429219~44.6488"
      ],
      "longitudeList": [
        "E09~-52.7126",
        "E10~-63.5752",
        "429219~-63.5752"
      ],
      "cataloguenumber": 4,
      "count": 2,
      "latitude": 47.5615,
      "longitude": -52.7126
    }
  ]
}
```

Figure 4: Example catalogue search response

3.3 Submissions Search

The submissions search endpoint handles dataset-level searches. This section lists its supported filters. It accepts four fields: last name, reference, reference DOI, and submission DOI. These match the Search by submission page on the web interface.

The logic was written in a dedicated API controller to avoid refactoring the original web files. It behaves identically to the web interface in terms of database queries, embargo

rules, and result structure. Results are aggregated to match the web interface's submission history view, with a count when multiple database rows collapse to the same signature. A SearchResult record is also stored to support the download workflow.

3.4 Download

The download endpoint handles file downloads. A single endpoint, POST `/api/v1/download`, accepts a `SearchResultId` in the request body. It is handled by `apiDownloadController` in `controllers/api/`. The controller loads the corresponding SearchResult record from the database and determines which packaging pipeline to invoke before generating a response.

Catalogue and submission searches are distinguished by the structure of the stored SearchResult. Catalogue searches store the originating query parameters under `InputTerms`. Submission searches do not. This distinction is used to route the request. If `InputTerms` is present and non-empty, the download is passed to the existing catalogue handler, `searchController.downloadSearchResult`. Otherwise, it is passed to `apiSubmissionDownloadController`, a submission download handler implemented specifically for the API. The submission download handler follows the same general approach as the website's submission download workflow. It uses the identifier list stored on the SearchResult to locate the relevant submission records and assemble an archive.

The two download types do not produce identical archive contents, but both include a tabular file that identifies the downloaded data.

The catalogue download ZIP contains a metadata CSV describing the matched specimens, a submission references CSV, a search terms text file, a ZIP of raw spectral files, and individual README text files for each submission. This matches the web catalogue download.

The submission download ZIP contains a submission information CSV and a set of per-submission archives. The submission information CSV is the metadata file for that download. It records fields such as reference, submission DOI, reference DOI, and type of data, so a user can tell what each packaged submission is from. Each per-submission archive then holds that submission's metadata file, raw spectral files, and README when those files exist. This matches the web submission download workflow.

For catalogue queries returning a large number of records, archive generation can take several minutes before the response is returned to the client.

3.5 Dataset Metadata

Separately from the CSV files packed inside a download ZIP, the API also provides a dataset metadata endpoint. This endpoint does not create an archive. It returns structured metadata for one dataset as JSON, so a client can inspect a submission without downloading its files.

Either a submission-info document ID or a submission ID can be provided in the path. The corresponding metadata is returned as plain JSON. This is useful when a script needs details for a known ID, whereas the identifying CSV files described in Section 3.4 travel with a bulk download of search results.

3.6 Validation and Error Handling

Submission search parameters are validated for minimum length and empty combinations, with HTTP 400 responses where appropriate (Fielding et al., 2022). Structured 400 responses apply where the API controllers explicitly validate input. This includes submission search and dataset metadata when the path identifier is absent.

Catalogue geographic parameters follow the same parsing path as the web search. Invalid geometries are not guaranteed to produce HTTP 400 unless additional validation is added

in a later iteration. Where validation exists, it improves client feedback and reduces the risk of malformed requests reaching the database layer.

Because the API does not fully validate polygon geometry, the R client performs its own client-side validation of polygon coordinates before a catalogue search request is sent.

Part B describes that validation in detail.

Part B: R Client (naturepaletteR)

Part B describes the implementation of the R package introduced in 2.4. It covers the package's internal structure, the object returned by a search, the behaviour of each exported function, and how the package is distributed.

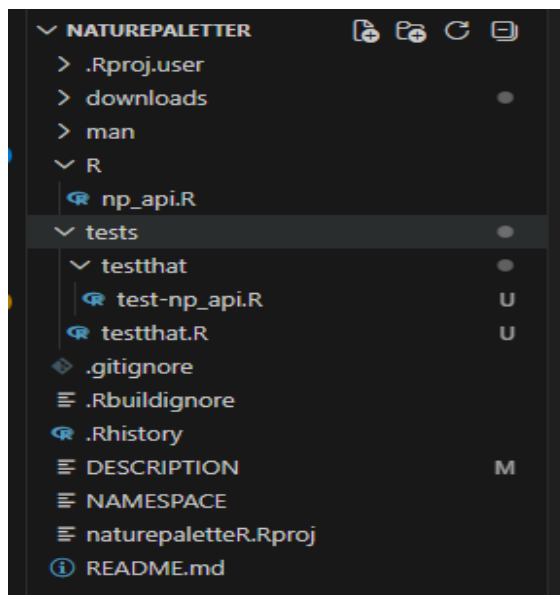


Figure 5: naturepaletteR package folder structure

3.7 Package Structure and Dependencies

`naturepaletteR` is a standard R package with a single source file, `R/np_api.R`. It contains all five exported functions, the S3 method `summary.np_search()`, and two internal helper functions. A single test file, `tests/testthat/test-np_api.R`, contains all automated tests. This structure keeps the package implementation centralized.

The package follows the standard R package layout, with `DESCRIPTION`, `NAMESPACE`, `README.md`, the `R/` directory, and the `tests/` directory. `DESCRIPTION` lists two runtime dependencies. `httr2` (Wickham, 2025) is used for all HTTP requests, and `jsonlite` (Ooms, 2014) is used to parse JSON responses from the API. `testthat` is listed under `Suggests` only, since it is required for development testing but not for end users. The package requires R 4.0.0 or later (R Core Team, 2024) and is released under the MIT licence.

3.8 The `np_search` Object

Both search functions return the same object type, a list with S3 class `np_search`. Table 3.8 summarizes its fields.

Table 3.8: Fields of the *np_search* object

Field	Type	Description
type	character	"catalogue" or "submission"
query	list	The query parameters passed by the user
base_url	character	API base URL used for the request
searchResultId	character	ID from the API, required by <code>np_download()</code>
totalMatchCount	numeric	Total number of matches
results	data.frame	Grouped search results table

As noted in 2.4, using one object class for both search types allows `summary()` and `np_download()` to work identically regardless of which function produced the object.

The results field is a plain `data.frame`, not a tibble. This avoids adding the tibble package as a dependency. The API response includes several internal MongoDB fields that are not meaningful outside the server. These include `SearchResultId`, `_id`, `rawFileId`, `latitudeList`, and `longitudeList`. They are removed before results are returned to the user. For catalogue searches, results contains columns relevant to the search, including `genus`, `specific epithet`, `infraspecific epithet`, `sex`, `lifestage`, `patch`, `individualCount`, `count`, `latitude`, and `longitude`. The API field `cataloguenumber` is renamed to `individualCount` in the R output.

3.9 Catalogue and Submission Search

`np_search_catalogue()` searches the metadata catalogue and corresponds to the website's Search data page. It sends a GET request to `/api/v1/catalogue/search` with the user's query list encoded as URL query parameters. Supported query fields correspond to the website search terms. These include genus, specific epithet, infraspecific epithet, sex, lifestage, patch, uniqueid, class, order, family, and other terms enabled in the repository's configuration, along with the geographic fields described in 3.10.

Internally, the function checks whether `query$polyCoordinates` is present. If it is, the polygon is validated and a matching bounding box is computed and added to the query automatically before the request is sent. The request is built and sent with `httr2`. The JSON body is parsed with `jsonlite`, and the response's success flag is checked. If success is false, the function stops with an error rather than returning a partially formed object. If the request succeeds, internal MongoDB columns are stripped from the results and `cataloguenumber` is renamed to `individualCount`. An `np_search` object with `type = "catalogue"` is returned.

`np_search_submissions()` searches by submission details and corresponds to the website's Search by submission page. It sends a GET request to `/api/v1/submissions/search`. At least one of four fields must be supplied, each with a minimum of three characters: `lastname`, `reference`, `referencedoi`, and `submissiondoi`. The function follows the same

request, parse, and error-checking pattern as `np_search_catalogue()`. It returns an `np_search` object with `type = "submission"`.

3.10 Polygon Search and Client-Side Validation

As described in 3.2, catalogue search requires a `polyCoordinates` parameter and a matching `mapCoordinate` bounding box for the API to return grouped results correctly. Polygon support in the R package required additional work to satisfy this requirement.

When polygon search was first integrated, sending only `polyCoordinates` produced the same empty-results behaviour described in 3.2. Inspection of the website's own network requests showed how the map search page constructs the `mapCoordinate` parameter alongside the drawn polygon. Since altering the API's behaviour was out of scope, and the existing contract already worked correctly for the website, the fix was implemented entirely on the R side. Two internal functions were added.

`validate_poly_coordinates()` checks that a polygon string is well formed before it is sent to the server. It confirms the string is not empty. It confirms the string contains an even number of values representing latitude and longitude pairs, and that it contains at least three distinct points. It confirms all values are numeric. It confirms latitudes fall between -90 and 90 and longitudes between -180 and 180. It confirms the points are not all

identical or duplicated consecutively. If the first and last point differ, the function closes the polygon by repeating the first point at the end, matching how the website forms polygon strings. It stops with a descriptive error message if any check fails.

`map_coordinate_from_polygon()` computes the `mapCoordinate` bounding box string from a validated polygon, in the format `maxLat_maxLng_minLat_minLng`, matching the format the website sends. When a user supplies `polyCoordinates` in a call to `np_search_catalogue()`, the package validates the polygon, computes the matching `mapCoordinate`, and adds both to the outgoing query automatically. The user only needs to provide the polygon.

Two ways to supply a polygon are supported. A string can be copied directly from the website, for example by drawing a shape on the map and reading the resulting coordinate string from the browser console. Alternatively, `np_format_polygon()` builds the string from a `data.frame` or matrix with `lat` and `lng` columns. It produces an underscore-separated string of the form `lat1_lng1_lat2_lng2_lat3_lng3`, removing duplicate consecutive points and requiring at least three distinct points before validation.

As noted in 3.6, this validation is client-side only and does not change the API's own behaviour.

```

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> library(naturepaletteR)
> pts <- data.frame( lat = c(2.08, -0.99, -1.17),
+ lng = c(-67.90, -65.13, -68.47))
> poly <- np_format_polygon(pts)
> s <- np_search_catalogue(query = list(genus = "Trogon",
+ polyCoordinates = poly))
> s
$type
[1] "catalogue"

$query
$query$genus
[1] "Trogon"

$query$polyCoordinates
[1] "2.08_-67.9_-0.99_-65.13_-1.17_-68.47_2.08_-67.9"

$query$mapCoordinate
[1] "2.08_-65.13_-1.17_-68.47"

$base_url
[1] "http://134.87.9.138/api/v1"

$searchResultId
[1] "6a58be8c805752cfdb9d01ff"

$totalMatchCount
[1] 0

$results
named list()

attr(,"class")
[1] "np_search"
> pts2 <- data.frame( lat = c(100.08, -21.12, 14.50),
+ lng = c(-270.50, 62.00, -210.10))
> poly2 <- np_format_polygon(pts2)
Error in validate_poly_coordinates(poly_str) :
  Latitude must be between -90 and 90.
>

```

Figure 6: Terminal output showing a polygon search and a validation error message for a malformed polygon

3.11 Download, Metadata, and summary()

`summary.np_search()` is an S3 method for the base R `summary()` generic, called as `summary(s)`. It prints a short overview of a search result: the search type, the query parameters used, the total match count, and a preview of the first few rows of results using `head()`. It applies to output from either search function, since both return the same `np_search` class.

`np_download()` downloads the ZIP archive associated with a previous search, whether catalogue or submission. It sends a POST request to `/api/v1/download` with a JSON body containing the `searchResultId` stored on the `np_search` object. The function first checks that `searchResultId` is present and stops with an error if it is missing. The output directory given in `path` is created if it does not already exist. The file is written using `filename`, which defaults to `natures_palette_download.zip`.

The HTTP request uses a 600 second timeout, since catalogue downloads for broad queries were observed to take several minutes on the live server. This limit was sufficient for the downloads tested in this project, but very large archives may need a longer or configurable timeout in a later release. The raw ZIP bytes returned by the server are written directly on the disk. The function prints short progress messages so that a user waiting on a long download has some indication the request is still running. It returns the full path to the saved file.

`np_download()` writes the ZIP archive to disk and does not extract its contents automatically. Archive contents differ between catalogue and submission downloads, as described in 3.4. The user extracts the archive manually.

`np_dataset_metadata()` retrieves metadata for a single dataset by ID. It sends a GET request to `/api/v1/datasets/{id}/metadata` and returns the parsed JSON response as a list. If the API's success flag is not true, for example when an unknown ID returns a 404 response, the function stops with an error rather than returning a partial or empty result.

3.12 Distribution and Usage

As described in 2.4 and 2.6, `naturepaletteR` is distributed through GitHub and installed with `remotes::install_github()`. Once installed, the package is loaded with `library(naturepaletteR)`.

A typical workflow runs a catalogue or submission search, inspects the result with `summary()`, and optionally calls `np_download()` to save the archive. Polygon search follows the same pattern when `polyCoordinates` is included in the catalogue query. The README documents installation, updates, and usage examples for each exported function, so a researcher can use the package without reading the source code directly. Automated tests for the R package are described in Chapter 4.


```

> library(naturepaletteR)
> s <- np_search_catalogue(query = list(genus = "Trogon", sex = "Male"))
> summary(s)
=== Nature Palette Search Summary ===

Search Type:
catalogue

Search Query:
$genus
[1] "Trogon"

$sex
[1] "Male"

Total Matches: 16

Catalogue Results Preview:
  genus specific epithet infraspecific epithet sex lifestage patch
1 Trogon                rufus                  Male      Adult  Head
2 Trogon                rufus                  Male      Adult  Head
3 Trogon                rufus                  Male      Adult  Head
4 Trogon                rufus                  Male      Adult  Head
5 Trogon                rufus                  Male      Adult  Head
  individualCount count latitude longitude
1                2     1      NA        NA
2                5     3      NA        NA
3               16     8 -2.6000 -56.73333
4                4     2      NA        NA
5                4     2 47.5615 -52.71260
> s2 <- np_search_submissions(query = list(lastname = "Sharier"))
> s2
$type
[1] "submission"

$query
$query$lastname
[1] "Sharier"

$base_url
[1] "http://134.87.9.138/api/v1"

$totalMatchCount
[1] 1

$searchResultId
[1] "6a58c59a805752cfdb9d021d"

$results
      submissionId      name typeOfData systemGeneratedDoi
1 6a559bc5805752cfdb9d0046 MD GOLAM SHARIER Reflectance 10.80611/vka5-4k80
count
1      1

reference
1 Taylor, G. (2019). Shakespeare's Early Gothic Hamlet. Critical Survey, 31(1-2), 4-25.
  numberOfFiles embargo releaseDate metaDataFileId
1             16   FALSE   2026-07-14 6a559bc5805752cfdb9d0047

attr(,"class")
[1] "np_search"
> np_download(s, path = "downloads", filename = "data.zip")
Starting download request...
Response received. Writing ZIP file...
Saved to: downloads/data.zip
[1] "downloads/data.zip"
>

```

Figure 7: Terminal output showing `np_search_catalogue()`, `summary(s)`, `np_search_submissions()` and `np_download()`

Chapter 4: Testing, Evaluation, and Discussion

This chapter reports how the REST API and R package were tested. Section 4.1 covers the Node.js API test suite, and Section 4.2 covers the R package test suite. Later sections discuss comparison with the web interface, performance, limitations, and future work.

4.1 API Test Suite

Once the core API implementation was complete, a lightweight Node.js test runner was written to check that the server is reachable and that the main endpoints return correct, well-structured responses across valid queries, edge cases, and invalid inputs. The suite started with `npm run api:test`.

The suite begins with a health check against `/ping`, expecting a 200 status and a body of `pong!`. Catalogue search tests then target `GET /api/v1/catalogue/search` and cover eight sub-cases, including basic search, OR and NOT queries, combined filters, case-insensitive matching, empty results, invalid syntax, and additional filter behaviour. Submission search tests target `GET /api/v1/submissions/search` and cover a basic seed query plus an invalid short input, which should return a non-200 response with success: `false`.

The download group covers both catalogue and submission paths. It first runs a seed search to obtain a searchResultId, then posts to /api/v1/download and validates the returned archive, checking for a 200 status, a body of at least 100 bytes, and the ZIP file signature (PK). The archive is then opened and checked for at least one non-directory entry, with byte size, file count, and a sample of entry names recorded in the report. This is usually the slowest group, since archive generation for broad catalogue queries can take several minutes.

The suite uses default configuration values, including the base URL `http://134.87.9.138/api/v1` and the seed queries, and these can be overridden with environment variables when needed. After each run, a timestamped report is saved to `api-tests/reports/`, listing each test group, its sub-cases, pass or fail results, and a final summary. The dataset metadata endpoint is implemented but is not yet part of this automated suite.

```
PS E:\Thesis\development - tpturjo\nature-palette---prototype-version\src> npm run api:test

> npsexpress@0.0.0 api:test
> node api-tests/run.js

BASE_URL: http://134.87.9.138/
catalogueQuery: {"genus":"Trogon"}
submissionsQuery: {"lastname":"Sharier"}

PASS: health
PASS: catalogue
PASS: submissions
PASS: download

Summary: 4 passed / 0 failed
Report: E:\Thesis\development - tpturjo\nature-palette---prototype-version\src\api-tests\reports\report_20260716_095717.txt
PS E:\Thesis\development - tpturjo\nature-palette---prototype-version\src> |
```

Figure 8: API test report showing pass/fail summary

4.2 R Package Test Suite

An automated test suite for `naturepaletteR` was written using `testthat` (edition 3) (Wickham, 2011). Where the API tests check endpoint behaviour, the R tests check that the wrapper functions call those endpoints correctly and return objects of the expected structure. Tests are run with `devtools::test()` during development and are included in the GitHub repository for maintenance, and end users installing the package do not run them.

All tests are kept in `tests/testthat/test-np_api.R`, organized into two layers. Offline tests do not require the API and cover polygon string construction, rejection of malformed polygons, automatic polygon closing and bounding box computation, and the `np_download()` check that stops when `searchResultId` is missing. Live API integration tests call the deployed server and cover catalogue search, polygon search with automatic `mapCoordinate` handling, submission search, `summary()` output, download and file save, and an error response for an unknown dataset ID. Live tests use `skip_if_api_down()`, so they skip rather than fail if the server is unreachable.

The live tests use fixed seed queries: catalogue search uses `genus = "Trogon"`, submission search and download use `lastname = "Sharier"`, and the metadata error test uses the fabricated ID `NP_DOES_NOT_EXIST_999`. The download test skips if the Sharier search returns no results, since database contents may change over time.

The suite covers nine test groups and twenty-one individual checks. A full run is expected to report [FAIL 0 | WARN 0 | SKIP 0 | PASS 21] when the server is reachable and the seed submission search returns at least one result. Like the API suite, live tests check structure and error handling rather than asserting that specific records always exist.

```

R 4.5.3 · C:/Users/tptur/Documents/naturepaletteR/
> devtools::test()
i Testing naturepaletteR
✓ | F W S OK | Context
:: |          17 | np_api

Starting download request...
Response received. Writing ZIP file...
Saved to: C:/Users/tptur/AppData/Local/Temp/RtmpCIxBnI/file13ac6f082ea6.zip

✓ |          21 | np_api [2.4s]

== Results ==
Duration: 2.4 s

[ FAIL 0 | WARN 0 | SKIP 0 | PASS 21 ]
> |

```

Figure 9: testthat summary showing PASS/FAIL/SKIP counts

4.3 Comparison with the Web Interface

To check that the REST API and R package return the same search behaviour as the existing website, the same queries were run against three interfaces: the Nature's Palette web UI, the /api/v1 endpoints on the production host (<http://134.87.9.138>), and the naturepaletteR client.

Two queries were inspected in detail with side-by-side outputs. Figure 10 shows a catalogue search for genus Trogon, sex Male, specific epithet rufus, and life stage Juvenile. Figure 11 shows a submission search by last name Sharier. In both cases, the website, API, and R client agreed on the match count and on the identifying attributes of the returned records.

Displaying 1 Unique Result (Found 2 in Total)								
Download								
#	Genus	Species	Subspecies	Sex	Life stage	Patch	Number of individuals	Number of files
1	Trogon	rufus	rufus	Male	Juvenile	Head	4	2

(a) the Nature's Palette web interface

```
{
  "success": true,
  "searchResultId": "6a58fe76805752cfdb9d0293",
  "totalMatchCount": 2,
  "results": [
    {
      "SearchResultId": "6a58fe76805752cfdb9d0293",
      "TotalSearchResultCount": 2,
      "_id": "6a559bc9805752cfdb9d0067",
      "rawFileId": "6a559bc5805752cfdb9d0053",
      "genus": "Trogon",
      "specificity": "rufus",
      "infraspecificity": "rufus",
      "sex": "Male",
      "lifestage": "Juvenile",
      "patch": "Head",
      "latitudeList": [
        "E09-47.5615",
        "E10-44.6488",
        "429219-44.6488"
      ],
      "longitudeList": [
        "E09-52.7126",
        "E10-63.5752",
        "429219-63.5752"
      ],
      "cataloguenumber": 4,
      "count": 2,
      "latitude": 47.5615,
      "longitude": -52.7126
    }
  ]
}
```

(b) the /api/v1 JSON response

```
> library(naturepaletteR)
> s <- np_search_catalogue(query = list(genus = "Trogon",
+ sex = "male", specificity = "rufus",
+ lifestage = "Juvenile"))
> s
$type
[1] "catalogue"

$query
$query$genus
[1] "Trogon"

$query$sex
[1] "male"

$query$specificity
[1] "rufus"

$query$lifestage
[1] "Juvenile"

$base_url
[1] "http://134.87.9.138/api/v1"

$searchResultId
[1] "6a58fffc3805752cfdb9d0297"

$totalMatchCount
[1] 2

$results
  genus specificity infraspecificity sex lifestage patch
1 Trogon      rufus          rufus Male Juvenile Head
  individualCount count latitude longitude
1              4    2  47.5615  -52.7126

attr(,"class")
[1] "np_search"
```

(a) the naturepaletteR console output

Figure 10: Catalogue search for genus Trogon, sex Male, specific epithet rufus, life stage Juvenile

Displaying 1 Unique Results (Found 1 in Total)							
Download selected submissions							
No.	Submission DOI	Reference	Data type	Number of Files	Embargo	Release Date	Download
1	10.88611/vka5-4k80	Taylor, G. (2019). Shakespeare's Early Gothic Hamlet. Critical Survey, 31(1-2), 4-25.	Reflectance	16	No	2026-07-14	

(a) the Nature's Palette web interface

```

Pretty-print
{
  "success": true,
  "searchResultId": "6a59022a805752cfdb9d029f",
  "totalMatchCount": 1,
  "results": [
    {
      "SearchResultId": "6a59022a805752cfdb9d029f",
      "TotalSearchResultCount": 1,
      "id": "6a559bc5805752cfdb9d004a",
      "submissionId": "6a559bc5805752cfdb9d0046",
      "name": "MD GOLAN SHARIER",
      "typeOfData": "Reflectance",
      "systemGeneratedDOI": "10.88611/vka5-4k80",
      "count": 1,
      "reference": "Taylor, G. (2019). Shakespeare's Early Gothic Hamlet. Critical Survey, 31(1-2), 4-25. ",
      "numberOfFiles": 16,
      "embargo": false,
      "releaseDate": "2026-07-14",
      "metaDataFileId": "6a559bc5805752cfdb9d0047"
    }
  ]
}

```

(b) the /api/v1 JSON response

```

> s2 <- np_search_submissions(query = list(lastname = "Sharier"))
> s2
$type
[1] "submission"

$query
$query$lastname
[1] "Sharier"

$base_url
[1] "http://134.87.9.130/api/v1"

$totalMatchCount
[1] 1

$searchResultId
[1] "6a59022a805752cfdb9d02a1"

$results
  submissionId      name typeOfData systemGeneratedDOI
1 6a559bc5805752cfdb9d0046 MD GOLAN SHARIER Reflectance 10.88611/vka5-4k80
count
1      1

1 Taylor, G. (2019). Shakespeare's Early Gothic Hamlet. Critical Survey, 31(1-2), 4-25.
  numberOfFiles embargo releaseDate      metaDataFileId
1             16 FALSE 2026-07-14 6a559bc5805752cfdb9d0047

attr(,"class")
[1] "np_search"

```

(a) the naturepaletteR console output

Figure 11: Submission search by last name Sharier

To broaden the comparison, additional catalogue and submission queries were run on all three interfaces and compared by match count. These included single-field catalogue searches such as genus, sex, specific epithet, life stage, patch, and infraspecific epithet, as well as further submission searches by last name, reference, and submission DOI. Some queries returned large result sets, for example genus Trogon (4666) and sex Male (5044). Table 4.3 summarizes all parity checks. Across every listed query, the web, API, and R totals agreed.

Table 4.3: Summary of parity checks

Query	Web	API totalMatchCount	R totalMatchCount
Catalogue: genus = Trogon	4666	4666	4666
Catalogue: sex = Male	5044	5044	5044
Catalogue: specificipithet = pusillus	126	126	126
Catalogue: lifestage = subadult	220	220	220
Catalogue: patch = Mantle	860	860	860
Catalogue: infraspecificipithet = amazonicus	438	438	438
Catalogue: Trogon / Male / rufus / Juvenile	2	2	2
Submissions: lastname = Sharier	1	1	1
Submissions: reference = Zoological Journal of the Linnean Society	1	1	1
Submissions: submissiondoi = 10.80611/txgw-1r63	1	1	1
Submissions: lastname = Bitton	2	2	2

Taken together, the detailed side-by-side checks and the broader match-count comparison support the design goal set out in Chapter 2, that the API and R client reuse existing search behaviour rather than introducing a separate result set.

4.4 Performance Observations

Response times were measured informally from a client machine against the production deployment (<http://134.87.9.138>). Each search was executed three times, and Table 4.4 lists all three runs and the median elapsed time in milliseconds. These figures are indicative only, since they depend on network conditions and server load and are not the result of formal benchmarking.

For the catalogue query (genus Trogon, sex Male, specific epithet rufus, life stage Juvenile), the API returned a median response time of 139 ms, measured with PowerShell. The corresponding web request, the browser's POST /search document request, had a median of 193 ms, and the R call `np_search_catalogue()` had a median of 200 ms, measured with `system.time()`. For the submissions query (last name Sharier), the API responded with a median of 118 ms, the web's GET searchSubmission/search document request had a median of 125 ms, and `np_search_submissions()` had a median of 110 ms.

These three measurements are not directly comparable byte-for-byte, since the API returns JSON only, the web measurement includes the full HTML document request and server-side rendering of the results page, and the R measurement includes both the HTTP round trip and parsing the response into an `np_search` object. Even so, all three stayed within a similar range for both queries, generally between 110 and 200 ms. A baseline GET /ping check, which does no search work, had a median of 156 ms, suggesting that a meaningful part of the observed time is network and connection overhead rather than query processing itself, at least for result sets this small.

First runs in a sequence were occasionally slower than the reported medians. An initial catalogue call reached 376 ms via the API and 620 ms in R, and an initial submissions call in R reached 230 ms, consistent with cold-start or connection setup. The median dampens that effect.

For these two test queries, all three interfaces completed within a few hundred milliseconds. This suggests the API and R client are suitable for interactive, scripted workflows, with the web UI carrying some additional cost from page rendering. No systematic performance testing beyond these spot checks was carried out.

Table 4.4: Response times (ms) for representative searches. Three runs per condition; median shown in the last column

Query	Interface	Run 1	Run 2	Run 3	Median
Catalogue (Trogon, Male, rufus, Juvenile)	API	376	139	121	139
Catalogue (Trogon, Male, rufus, Juvenile)	Web	192	294	193	193
Catalogue (Trogon, Male, rufus, Juvenile)	R	620	160	200	200
Submissions (lastname = Sharier)	API	213	117	118	118
Submissions (lastname = Sharier)	Web	116	125	188	125
Submissions (lastname = Sharier)	R	230	110	110	110
API ping (baseline)	API	156	201	117	156

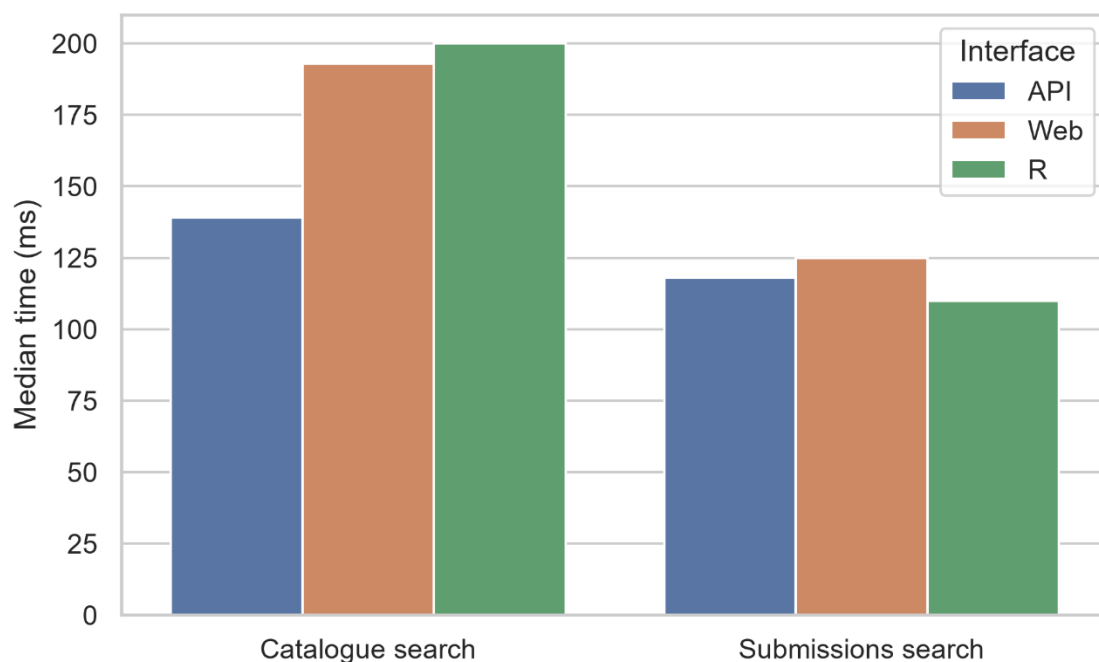


Figure 12: Median response times for two search queries on the production server, by interface (API, web document request, R client). Derived from Table 4.4; indicative measurements, not a formal benchmark

4.5 Limitations

Several limitations apply to this evaluation and to the scope of the API and R interface as delivered.

Evaluation depth: Functional parity between the web UI, REST API, and R client was checked in detail for two representative queries, shown in Figures 10 and 11, and more broadly by match-count comparison for the additional queries listed in Table 4.3, including large catalogue result sets. This demonstrates the design goal of mirroring legacy search behaviour for typical cases, but it is not exhaustive verification. The

automated suite (`npm run api:test`) exercises broader catalogue logic, including OR/NOT combinations and both download paths, but neither it nor the manual comparison covers every field combination, geographic filter shape, or operator combination. A full user study with external researchers was outside the scope of this Honours project.

Performance: The timings in Table 4.4 and Figure 12 are informal spot checks, three runs per condition, from a single client machine, over one network path to the production host. They are not a controlled benchmark, since there was no repetition across time of day, no concurrent load, and no isolation of server-side from network-side latency. The three interfaces also measure different work, since the API and R retrieve and parse JSON while the web interface retrieves and renders HTML. The medians in Figure 12 should therefore be read as indicative rather than as a strict ranking.

Security and access control: The API endpoints under `/api/v1` are open by design, matching the existing public website. There is no API key, token, or per-organization authentication, such as the OAuth 2.0 authorization framework (Hardt, 2012), and no rate limiting is enforced at the API boundary. This mirrors the scope decision in Section 1.5 to add programmatic access without altering the repository's public data policy, and it keeps the barrier to entry low. However, it also means the API cannot distinguish a trusted integration from arbitrary traffic, has no per-client quotas, and keeps no audit trail. On shared infrastructure, an unbounded client script could place a disproportionate load on

the server compared with normal web traffic, since search and download operations are heavier than a typical page request. This was a deliberate scope decision rather than an oversight, and it is addressed as future work below.

System scope: The project adds programmatic search and download. It does not alter the submission, curation, or validation pipeline, and it introduces no new biological query fields beyond those already searchable on the website. The API reuses legacy controller logic where possible, so any existing quirk in web search behaviour is inherited rather than independently re-verified.

Deployment and dependencies: Results depend on the specific deployed server (134.87.9.138 at evaluation time) and on the naturepaletteR version published on GitHub at that time. No formal version pinning exists between the web application, the /api/v1 routes, and the R client, so behaviour could drift if one component is updated independently of the others. Operational concerns such as uptime monitoring, scaling, and failover were not part of this project's deliverables.

R package maturity: naturepaletteR is distributed through GitHub rather than CRAN, lacks exhaustive roxygen documentation, and has not been evaluated with external domain scientists within the project timeline.

4.6 Future Work

Extensions to this work fall into four areas: access control, testing and reliability, documentation and distribution, and performance.

Access control and governance: The most direct follow-on to the limitation above would be introducing optional API keys or a lightweight OAuth-style credential, such as one based on the OAuth 2.0 authorization framework (Hardt, 2012), paired with per-client rate limiting. This need not restrict the underlying open data policy, since public results could remain freely accessible, but it would let the server throttle misbehaving clients and provide an audit trail, addressing the gap noted in Section 4.5 without changing who can access the data.

Testing and continuous integration: The existing suite (`npm run api:test`) could be wired into continuous integration so checks run automatically on every deployment rather than manually. The parity checks in Section 4.3, including the match-count comparisons in Table 4.3, could also be turned into a version-controlled regression set, using fixed queries with expected `totalMatchCount` values or known identifiers, so that future changes to either the web application or the API can be checked for agreement without repeating manual screenshots.

Documentation and distribution: Publishing an OpenAPI specification would make `/api/v1` easier for external developers to discover and would complement the existing R package documentation, in keeping with the broader goal of making research data and interfaces findable and reusable (Wilkinson et al., 2016). For `naturepaletteR`, moving toward a CRAN submission or adding runnable vignettes covering the search, download, and analyze workflow would lower the adoption barrier for researchers not already comfortable installing packages from GitHub.

Performance engineering: A follow-up study could replace the informal measurements in Table 4.4 with a controlled benchmark, using repeated trials across different times of day, server-side timing isolated from network latency, and testing under concurrent requests. This would help clarify whether the variability seen in the current three-run samples, including occasional slow first requests likely caused by connection warm-up, reflects genuine server-side cost or client and network noise.

API evolution and alignment: As the web application gains new search fields, the corresponding API controllers should be updated to keep programmatic and interactive access consistent with the original mirroring principle. If request volume grows, introducing pagination through limit and offset parameters and a versioned `/api/v2`, while keeping `/api/v1` stable for existing scripts, would let the interface evolve without breaking

current integrations. Basic hardening such as HTTPS enforcement, security headers, and request logging would pair naturally with any future authentication layer.

Chapter 5: Conclusion

5.1 Summary of Work

This project added programmatic access to the Nature's Palette spectral data repository, which previously supported only interactive use through a web interface. Two components were delivered: a REST API mounted under `/api/v1` in the existing Node.js, Express, and MongoDB application, and an R package, `naturepaletteR`, that wraps those endpoints for use in scripted research workflows.

The API exposes catalogue search, submission search, dataset metadata retrieval, and dataset download, with controllers that reuse the existing web application's search and download logic so that programmatic results mirror the website rather than introducing separate query behaviour. The R package exposes this functionality through functions such as `np_search_catalogue()` and `np_search_submissions()`, returning results as structured `np_search` objects that fit naturally into R-based analysis. The design followed one guiding principle throughout, which was to extend the system additively without breaking the live website or altering the submission and validation pipeline.

5.2 Evaluation Outcomes

The API and R package were checked with automated tests. The Node.js API suite exercised connectivity, catalogue search, submission search, and download, while the R package suite used `testthat` to check that the wrapper functions call the endpoints correctly and return well-structured objects.

Functional parity was checked across the web interface, the API, and the R client. Two queries were inspected in detail with side-by-side outputs, and additional catalogue and submission queries were compared by match count, including larger result sets. The three interfaces agreed on the reported totals. Informal performance observations showed all three interfaces returning results in well under a second for the timed test queries, and a baseline health check suggested that network round-trip accounted for a meaningful part of that time on small result sets. These checks were deliberately scoped rather than exhaustive, consistent with the limitations discussed in Chapter 4.

5.3 Contributions

The main contributions of this project are:

- A versioned REST API (`/api/v1`) providing programmatic search and download for Nature's Palette, built to mirror existing web behaviour.

- An R package, `naturepaletteR`, that makes the repository accessible from a common scientific computing environment, returning analysis-ready objects.
- An automated API test suite and a documented parity and performance evaluation demonstrating consistency between the programmatic and web interfaces.

Together these let researchers define searches in code, rerun them as the repository grows, and share reproducible retrieval scripts, capabilities the web interface alone could not provide.

5.4 Concluding Remarks

This work turned a browser-only repository into one that also supports reproducible, programmatic access, while leaving the existing website and data intact. The service was scoped deliberately and kept open, consistent with the existing public access model, and the remaining gaps described in Sections 4.5 and 4.6 point to the practical next steps for anyone extending this work. Even in its current form, the API and R interface provide a working foundation for integrating Nature's Palette data directly into scientific analysis.

References

- Akhter, R. (2020). Nature's Palette – An open access repository of spectral data [Master's thesis, Memorial University of Newfoundland].
- Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force.
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].
- Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force.
- Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6749>
- Kamal, S. I. (2020). Nature's Palette Validator [Honours thesis, Memorial University of Newfoundland].
- Khan, M. H. (2023). Color space filtering in Nature's Palette [Project report, Memorial University of Newfoundland].
- Maia, R., Eliason, C. M., Bitton, P.-P., Doucet, S. M., & Shawkey, M. D. (2013). pavo: An R package for the analysis, visualization and organization of spectral data. *Methods in Ecology and Evolution*, 4(10), 906–913. <https://doi.org/10.1111/2041-210X.12069>
- Norton, B. (2021). APIs: a common interface for the global biodiversity informatics community. *Biodiversity Information Science and Standards*, 5.
- Ooms, J. (2014). The jsonlite package: A practical and consistent mapping between JSON data and R objects. *arXiv:1403.2805 [stat.CO]*.
- Powers, S. M., & Hampton, S. E. (2019). Open science, reproducibility, and transparency in ecology. *Ecological Applications*, 29(1), e01822.
- R Core Team. (2024). R: A language and environment for statistical computing. R Foundation for Statistical Computing.
- Reichman, O. J., Jones, M. B., & Schildhauer, M. P. (2011). Challenges and opportunities of open data in ecology. *Science*, 331(6018), 703–705.
- Telenius, A. (2011). Biodiversity information goes public: GBIF at your service. *Nordic Journal of Botany*, 29(3), 378–381.

Wickham, H. (2011). testthat: Get started with testing. *The R Journal*, 3(1), 5–10.

Wickham, H. (2025). http2: Perform HTTP requests and process the responses (R package version 1.2.2) [Computer software].

Wieczorek, J., Bloom, D., Guralnick, R., Blum, S., Döring, M., et al. (2012). Darwin Core: An evolving community-developed biodiversity data standard. *PLoS ONE*, 7(1), e29715.

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., ... & Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3(1), 1–9.